

Python E-Course

Module 1 — Introduction to Python

Detailed Notes

Module Overview

Level: Beginner

Duration: ~1 week (6-8 hours)

Prerequisites: None — basic computer literacy is enough

What You'll Learn

By the end of this module, you will be able to:

- Explain what Python is and why it's widely used.
- Install Python 3 on Windows, macOS, or Linux.
- Use the Python REPL interactively.
- Write, save, and run a Python script.
- Read and write clean Python syntax (indentation, comments, statements).
- Choose and configure an IDE (VS Code, PyCharm, or others).

Lessons

#	Lesson	Duration
1	What is Python?	25 min
2	Installing Python & Setting Up Your Environment	30 min
3	Your First Python Program	25 min
4	Python Syntax Basics	35 min
5	Comments, Indentation & Code Style	25 min
6	Running Python: REPL, Scripts, IDEs	30 min

Assessment

- Exercises — 18 exercises (3 per lesson)
- Quiz — 10 MCQs
- Assignment — Personal Info Card program
- Mini Project — Interactive Calculator

Module Goals

After this module you'll be set up to learn: Python installed, an editor configured, a working mental model of "what running a Python program means," and enough syntax fluency to follow Module 2 without friction.

What's Next

Module 2: Variables & Data Types — store information, do math, take user input.

Lesson 1: What is Python?

Module: 1 — Introduction to Python

Duration: 25 minutes

Difficulty: Beginner

Learning Objectives

By the end of this lesson, you will be able to:

- Describe what Python is and what it's used for.
- Identify the differences between Python 2 and Python 3.
- Name at least three industries where Python dominates.
- Explain why Python is a good first language.

Prerequisites

- None.

1. Why This Matters

Before you spend weeks learning a language, it pays to know what you're getting and what you can do with it. Python is the most popular programming language in the world by most rankings — used by Google, Netflix, NASA, Instagram, and millions of solo developers. Understanding its strengths (and limits) helps you pick the right tool later, instead of forcing Python into jobs another language does better.

2. Concept

2.1 What Python Is

Python is a general-purpose, high-level, interpreted programming language. Let's unpack each term:

- General-purpose: you can build web apps, automate scripts, do data science, train machine learning models, write games, build desktop tools — all with Python.
- High-level: the language hides hardware details (memory, registers) so you focus on solving problems.
- Interpreted: Python code runs line-by-line through an interpreter, not compiled to a separate executable file. This makes development fast and feedback immediate.

Python was created by Guido van Rossum in 1989 and released in 1991. He named it after the British comedy Monty Python's Flying Circus — not the snake.

2.2 Python 2 vs Python 3

	Python 2	Python 3
Released	2000	2008

Status	End-of-life (Jan 2020)	Active, recommended
print	statement: <code>print "hi"</code>	function: <code>print("hi")</code>
Division	<code>5 / 2 == 2</code> (integer)	<code>5 / 2 == 2.5</code> (true)
Strings	bytes by default	Unicode by default

Always use Python 3. This course targets Python 3.10 or newer. If you see Python 2 tutorials online, skip them.

2.3 What Python Is Used For

Field	Examples
Web development	Instagram (Django), Reddit, Spotify backend
Data science	NumPy, pandas, Jupyter — the standard toolkit
Machine learning	TensorFlow, PyTorch, scikit-learn
Automation / scripting	DevOps glue code, system administration
Scientific computing	Physics, biology, astronomy research
Game development	EVE Online server, Civilization IV scripts
Education	Most universities now teach intro CS in Python

2.4 Why Python is a Good First Language

- Readable syntax: Python code looks close to English. `if x > 5: print("big")` reads naturally.
- Small barrier to running code: no compilation step, no boilerplate `main()` function required.
- Huge community: any error message you'll hit has been answered on Stack Overflow.
- Transferable: concepts you learn (variables, loops, functions, OOP) apply to every modern language.

2.5 Where Python Isn't the Best Choice

To set honest expectations:

- Mobile apps: Swift (iOS) or Kotlin (Android) are the standard. Python on mobile is niche.
- Highly performance-critical code: a tight numerical loop in Python is ~50x slower than the same loop in C. (Libraries like NumPy work around this by calling C under the hood.)
- Real-time systems with strict timing (embedded, audio DSP): typically C or Rust.

Python's response to performance gaps is "call faster code from Python" — you'll see this in Modules 10 and beyond.

3. Code Examples

Example 1: Python's signature readability

```
# Greet someone if they have a name
name = "Alex"
if name:
    print("Hello,", name)
else:
    print("Hello, stranger")
```

Expected Output:

```
Hello, Alex
```

Explanation: Notice how close this reads to plain English. You don't need to declare types, allocate memory, or write boilerplate to run a few lines.

Example 2: Same task in Python vs Java

```
# Python – print numbers 1 through 5
for i in range(1, 6):
    print(i)
```

Expected Output:

```
1
2
3
4
5
```

Explanation: The equivalent Java program is ~6 lines and requires a class wrapper. Python lets beginners focus on the idea (looping 1 to 5) without ceremony.

Example 3: Checking your Python version

```
import sys
print(sys.version)
```

Expected Output (example, your version may differ):

```
3.11.5 (main, Sep 7 2023, 12:00:00) [MSC v.1936 64 bit (AMD64)]
```

Explanation: Every Python install comes with a built-in sys module that exposes runtime info. You'll use this to confirm you're on Python 3.10+ in the next lesson.

4. Common Mistakes

Mistake	What Happens	Fix
Following a Python 2 tutorial	Code uses print "x" and won't run	Confirm the tutorial says Python 3
Confusing Python with PyPy or Jython	Different runtimes with different libraries	Stick to CPython (the default) from python.org
Assuming Python is "just for data science"	You miss its huge use in web, automation, scripting	It's general-purpose — apply it broadly

5. Practice Tasks

- Identify (2 min): Find one famous app or website built with Python (search "built with Python"). Note one feature it offers.
- Compare (5 min): Open the Python docs and the docs for another language you know (or have heard of). Compare how easy it is to find the "Tutorial" link from the homepage.

- Reflect (5 min): Write down two reasons you personally want to learn Python. You'll re-read this at the end of the course.

6. Key Takeaways

- Python is a high-level, interpreted, general-purpose language.
- Use Python 3 — Python 2 is end-of-life.
- It dominates data science, automation, and backend web development.
- It's a great first language because of its readable syntax and huge ecosystem.

7. What's Next

In the next lesson, we'll install Python 3 on your machine and confirm it runs — turning theory into a working environment.

Lesson 2: Installing Python & Setting Up Your Environment

Module: 1 — Introduction to Python

Duration: 30 minutes

Difficulty: Beginner

Learning Objectives

By the end of this lesson, you will be able to:

- Install Python 3.10 or newer on Windows, macOS, or Linux.
- Verify the install from the command line.
- Open and use the Python REPL.
- Install and open a code editor (VS Code).

Prerequisites

- Lesson 1: What is Python?
- A computer running Windows 10+, macOS 11+, or any modern Linux.
- Administrator/sudo privileges on the machine.

1. Why This Matters

A working Python install is the foundation for everything else in this course. Most beginners hit the same setup snags: not adding Python to PATH, installing the wrong version, or confusing the system Python with a project-specific one. Doing this carefully now saves hours of confusion later.

2. Concept

2.1 Which Python You Want

- CPython — the official, default Python from python.org. Use this.
- Anaconda — a heavy distribution bundling Python + data-science libraries. Useful later if you go into data science. Skip it for now.
- PyPy, Jython, IronPython — alternative interpreters. Not for this course.

Target version: Python 3.10 or newer. This course uses features (like match/case) that need 3.10+.

2.2 Installation Steps by OS

Windows

- Go to python.org/downloads.
- Click Download Python 3.x.x (latest stable).
- Run the installer.
- CRITICAL: Check the box "Add Python to PATH" before clicking Install.

- Click Install Now.

macOS

- Go to python.org/downloads.
- Download the macOS 64-bit universal installer.
- Run it and follow the prompts.

Alternative (recommended if you have it): brew install python3 using Homebrew.

Linux (Ubuntu/Debian)

Most distributions already have Python 3. To check or upgrade:

```
sudo apt update
sudo apt install python3 python3-pip
```

2.3 Verifying the Install

Open a terminal:

- Windows: press Win + R, type cmd, press Enter. (Or use PowerShell or Windows Terminal.)
- macOS: open Terminal.app from Applications → Utilities.
- Linux: use your distro's terminal.

Type:

```
python --version
```

If that doesn't work (Windows often needs the 3 suffix or py):

```
python3 --version
py --version
```

You should see something like Python 3.11.5. If you see Python 2.x or get "command not found", the install or PATH is broken — re-run the installer.

2.4 The Python REPL

REPL = Read-Eval-Print Loop. It's an interactive Python prompt where each line you type runs immediately.

Open it by typing python (or python3) in the terminal. You'll see:

```
>>> _
```

Type any Python expression and press Enter. To exit, type exit() or press Ctrl+D (macOS/Linux) / Ctrl+Z then Enter (Windows).

2.5 Installing a Code Editor

You can write Python in any text editor — even Notepad — but a real editor gives you syntax highlighting, auto-complete, and error squiggles. Recommended:

Editor	Notes
VS Code	Free, lightweight, huge extension ecosystem. Recommended for this course.
PyCharm Community	Free, heavier, opinionated Python-specific tooling. Good for larger projects.
Cursor / Zed	Modern alternatives if you want them.

To install VS Code:

- Go to code.visualstudio.com.
- Download and run the installer.
- After installing, open VS Code → click the Extensions icon (left sidebar, or Ctrl+Shift+X) → search for "Python" → install the one by Microsoft.

3. Code Examples

Example 1: REPL basic math

Start the REPL by typing `python` in your terminal, then enter:

```
>>> 2 + 2
>>> 10 / 4
>>> "hello".upper()
```

Expected Output:

```
4
2.5
'HELLO'
```

Explanation: The REPL prints results without needing `print()`. Great for quick experiments.

Example 2: Check the version from inside Python

In the REPL:

```
>>> import sys
>>> sys.version_info
```

Expected Output (your numbers may differ):

```
sys.version_info(major=3, minor=11, micro=5, releaselevel='final', serial=0)
```

Explanation: Confirms the exact Python you're running. The major field should be 3; minor should be 10 or higher.

Example 3: Verify pip is installed

In your terminal (not the REPL — exit it first with `exit()`):

```
pip --version
```

Expected Output (your path will differ):

```
pip 23.2.1 from C:\Python311\Lib\site-packages\pip (python 3.11)
```

Explanation: pip is Python's package installer — you'll use it from Module 10 onward to install third-party libraries.

4. Common Mistakes

Mistake	What Happens	Fix
Forgot "Add Python to PATH" on Windows	python is not recognized in terminal	Re-run installer → choose Modify → check PATH box
Installed Python 2 by mistake	Course examples fail	Uninstall it; reinstall from python.org (3.10+)
Using python when system has only python3	Command not found	Use python3 instead (common on macOS/Linux)
Confused about which Python an editor uses	Code runs with the wrong interpreter	In VS Code press Ctrl+Shift+P → "Python: Select Interpreter"
Installing Python from Microsoft Store on Windows	Path issues with some tools later	Prefer the python.org installer

5. Practice Tasks

- Install & verify (10 min): Install Python 3.10+ if you haven't. Run `python --version` and write down the output.
- REPL math (5 min): Open the REPL and calculate: (a) 2 to the power of 10, (b) the remainder of 17 divided by 5, (c) the length of the string "supercalifragilistic".
- Editor setup (10 min): Install VS Code and the Python extension. Open VS Code and find the Python interpreter selector in the status bar at the bottom.

6. Key Takeaways

- Install CPython 3.10+ from python.org.
- On Windows, check "Add Python to PATH" during install.
- Verify with `python --version` (or `python3 --version`).
- The REPL is for quick experiments; VS Code is for writing files.
- pip comes bundled with Python — you'll use it later.

7. What's Next

In the next lesson, we'll write and run your first complete Python program from a saved file — not just the REPL.

Lesson 3: Your First Python Program

Module: 1 — Introduction to Python

Duration: 25 minutes

Difficulty: Beginner

Learning Objectives

By the end of this lesson, you will be able to:

- Write a Python program in a .py file.
- Run a Python script from the terminal.
- Use the `print()` function to display output.
- Explain the difference between a statement and an expression.

Prerequisites

- Lesson 1: What is Python?
- Lesson 2: Installation & setup
- Python 3.10+ installed and verifiable with `python --version`.

1. Why This Matters

The REPL is great for one-liners, but real programs live in files. You need to know how to save code, run it, and see its output — because every project you build (assignments, mini-projects, the capstone) is a .py file you run from the command line.

2. Concept

2.1 A Python program is just a text file

A Python script is a plain text file ending in .py. The Python interpreter reads it top-to-bottom and executes each line in order.

You can write it in any text editor. We'll use VS Code (from Lesson 2).

2.2 The `print()` function

`print()` is Python's built-in function for displaying output. Whatever you put inside the parentheses gets shown on the screen:

```
print("Hello, World!")
```

You can print strings (text in quotes), numbers, results of calculations, or values stored in variables. Multiple arguments are separated by spaces by default:

```
print("Hello, ", "World!")
```

2.3 Statements vs. Expressions

- Expression: a piece of code that produces a value. `2 + 3` is an expression (value: 5). `"hi".upper()` is an expression (value: "HI").
- Statement: an instruction Python performs. `print("hi")` is a statement. `x = 5` is a statement.

Many statements contain expressions. `print(2 + 3)` is a statement containing the expression `2 + 3`.

In the REPL, expressions print their value automatically. In a file, they don't — you need `print()`.

2.4 How to run a Python file

Three options:

Option A — Terminal (works everywhere)

- Open a terminal.
- Navigate to the folder where your file is saved: `cd path/to/folder`.
- Run: `python hello.py` (or `python3 hello.py` on macOS/Linux).

Option B — VS Code Run button

- Open the `.py` file in VS Code.
- Click the triangle (▶) play button in the top-right.

Option C — IDLE (comes bundled with Python)

- Open IDLE (search for it in your Start menu / Applications).
- File → Open → pick your `.py` file.
- Press F5.

For this course, prefer Option A — it works for everyone and forces you to learn the terminal, which you need anyway.

3. Code Examples

Example 1: Hello, World!

Save this as `hello.py`:

```
print("Hello, World!")
```

Then in a terminal, from the folder containing it:

```
python hello.py
```

Expected Output:

```
Hello, World!
```

Explanation: This is the smallest meaningful Python program. The string "Hello, World!" is passed to `print()`, which displays it.

Example 2: A multi-line program

Save this as `intro.py`:

```
print("Welcome to Python.")
print("This is line two.")
print("And this is line three.")
```

Run with `python intro.py`.

Expected Output:

```
Welcome to Python.
This is line two.
And this is line three.
```

Explanation: Each `print()` call ends with a newline by default. Statements run in the order they appear.

Example 3: Printing numbers and math

Save as `math.py`:

```
print(2 + 3)
print(10 * 7)
print(100 - 1)
print(20 / 4)
```

Expected Output:

```
5
70
99
5.0
```

Explanation: Python evaluates the expressions inside `print()` before showing the result. Note that `/` always produces a float (5.0, not 5) — you'll see why in Module 2.

Example 4: Multiple arguments to `print()`

Save as `multi.py`:

```
print("Name:", "Alex")
print("Age:", 25, "years")
print("Score:", 9 + 6)
```

Expected Output:

```
Name: Alex
Age: 25 years
Score: 15
```

Explanation: `print()` accepts multiple comma-separated arguments and inserts a space between them.

Example 5: Using `sep` and `end`

Save as `format.py`:

```
print("apple", "banana", "cherry", sep=", ")
print("Loading", end="...")
print("done")
```

Expected Output:

```
apple, banana, cherry
Loading...done
```

Explanation: `sep` changes the separator between arguments (default: space). `end` changes what's printed after the last argument (default: newline `\n`).

4. Common Mistakes

Mistake	What Happens	Fix
Saving file as <code>.txt</code> instead of <code>.py</code>	Python doesn't recognize it; editor won't highlight syntax	Save as <code>something.py</code>
Running python in the wrong folder	can't open file 'hello.py': [Errno 2] No such file or directory	cd to the folder first, or pass the full path
Forgetting the parentheses: <code>print "hi"</code>	SyntaxError (Python 2 syntax)	Use <code>print("hi")</code>
Mixing quotes: <code>print("hi')</code>	SyntaxError: EOL while scanning string literal	Match <code>" "</code> or <code>' '</code>
Pressing Run on an unsaved file	Runs an older saved version	Save first (Ctrl+S)

5. Practice Tasks

- Hello You (2 min): Write a one-line program that prints Hello, [your name]!. Save as `hello_me.py` and run it.
- Three Facts (5 min): Write a program that prints three facts about yourself (one per line) using three `print()` calls.
- Math display (5 min): Write a program that prints the result of: (a) 17×23 , (b) 1000 minus 199, (c) the integer division of 100 by 7 (use `//` instead of `/`). Compare your output with a calculator.

6. Key Takeaways

- Python scripts are plain text files saved with the `.py` extension.
- Run them with `python filename.py` from the terminal.
- `print()` is your output tool — it takes one or more arguments, separated by `sep`, and ends with `end`.

- The interpreter reads top-to-bottom.

7. What's Next

In the next lesson, we'll look at Python's syntax rules — how statements, blocks, and lines work, and why indentation is special in Python.

Lesson 4: Python Syntax Basics

Module: 1 — Introduction to Python

Duration: 35 minutes

Difficulty: Beginner

Learning Objectives

By the end of this lesson, you will be able to:

- Identify a Python statement, expression, and identifier.
- Use indentation to define code blocks correctly.
- Continue long lines using implicit and explicit continuation.
- Recognize and avoid Python's reserved keywords.
- Read error messages and trace them to the offending line.

Prerequisites

- Lesson 3: Your first Python program
- Ability to save and run a .py file.

1. Why This Matters

Most beginner bugs are syntax errors — missing colons, wrong indentation, or invalid names. Once you can read Python syntax fluently, errors stop feeling random and start feeling fixable. This lesson is the rulebook.

2. Concept

2.1 Statements

A statement is one instruction. By default, statements end at the end of the line — no semicolons needed.

```
x = 5
print(x)
```

You can put multiple statements on one line with `;`, but don't — it's ugly and rare in real Python:

```
x = 5; print(x)  # legal but discouraged
```

2.2 Indentation Defines Blocks

Most languages use `{ ... }` or `begin/end` to group code. Python uses indentation. A block is any group of statements indented the same amount under a header line.

```
if 5 > 2:
    print("five is bigger")
    print("still inside the if")
print("outside the if")
```


Rules:

- Use 4 spaces per level (PEP 8 standard).
- Be consistent — don't mix tabs and spaces.
- The interpreter raises `IndentationError` if you get it wrong.

We'll see blocks formally in Module 3 (if statements, loops). For now, just remember: a line ending in `:` starts a new indented block.

2.3 Identifiers (Names)

An identifier is a name you give to a variable, function, class, etc. Rules:

- Must start with a letter (a-z, A-Z) or underscore `_`.
- After the first character, can contain letters, digits, and underscores.
- Case-sensitive: `score` and `Score` are different.
- Cannot be a Python keyword (see 2.4).

Valid: `name`, `total_score`, `_private`, `data2`, `userName`

Invalid: `2score` (starts with digit), `total-score` (hyphen), `class` (keyword), `my var` (space)

Convention (PEP 8): use `snake_case` for variables and functions — `total_score`, not `totalScore`.

2.4 Reserved Keywords

These 35 words are reserved by Python — you can't use them as variable names:

<code>False</code>	<code>None</code>	<code>True</code>	<code>and</code>	<code>as</code>	<code>assert</code>	<code>async</code>	<code>await</code>
<code>break</code>	<code>class</code>	<code>continue</code>	<code>def</code>	<code>del</code>	<code>elif</code>	<code>else</code>	<code>except</code>
<code>finally</code>	<code>for</code>	<code>from</code>	<code>global</code>	<code>if</code>	<code>import</code>	<code>in</code>	<code>is</code>
<code>lambda</code>	<code>nonlocal</code>	<code>not</code>	<code>or</code>	<code>pass</code>	<code>raise</code>	<code>return</code>	<code>try</code>
<code>while</code>	<code>with</code>	<code>yield</code>					

You can list them yourself:

```
import keyword
print(keyword.kwlist)
```

2.5 Line Continuation

Lines should be reasonably short (PEP 8 suggests max 79 characters). If you need to break a long line:

Implicit — inside `()`, `[]`, `{}` you can wrap freely:

```
total = (1 + 2 + 3 +
         4 + 5 + 6)
```

Explicit — use a backslash `\` at the end of the line:

```
total = 1 + 2 + 3 + \  
        4 + 5 + 6
```

Prefer implicit continuation. Backslashes are easy to break accidentally (a trailing space after \
causes a syntax error).

2.6 Case Sensitivity

Python is case-sensitive:

```
Name = "Alex"  
name = "Bob"  
print(Name)    # Alex  
print(name)    # Bob
```

Two completely different variables. Beginners hit this when they type Print(...) instead of
print(...) and get NameError: name 'Print' is not defined.

2.7 Reading Error Messages

Errors are how Python tells you what went wrong. Read them bottom-up:

```
print("hi"
```

Error:

```
File "test.py", line 1  
    print("hi"  
          ^  
SyntaxError: '(' was never closed
```

- Last line = type of error (SyntaxError) + brief explanation.
- ^ caret = approximately where Python noticed the problem.
- File and line = where to look.

Common error types you'll meet in this module:

Error	Means
SyntaxError	The code is not valid Python (often a typo)
IndentationError	Wrong or inconsistent indentation
NameError	Used a name that doesn't exist yet
TypeError	Used a value the wrong way (e.g. "5" + 3)

3. Code Examples

Example 1: Valid statements

```
greeting = "Hello"  
name = "Sam"  
print(greeting, name)
```

Expected Output:

```
Hello Sam
```

Explanation: Three statements, each on its own line. Variables are assigned with =, then used in print().

Example 2: Indentation defines a block

```
temperature = 30
if temperature > 25:
    print("It's hot.")
    print("Drink water.")
print("Done.")
```

Expected Output:

```
It's hot.
Drink water.
Done.
```

Explanation: The two indented print calls belong to the if block. The third print is at top level and always runs.

Example 3: IndentationError demo

```
if True:
    print("oops")
```

Expected Output:

```
File "demo.py", line 2
    print("oops")
    ^
IndentationError: expected an indented block after 'if' statement on line 1
```

Explanation: Python expected indented code after the if: header. Always indent the body of an if.

Example 4: Implicit line continuation

```
items = [
    "apple",
    "banana",
    "cherry",
    "date",
]
print(items)
```

Expected Output:

```
['apple', 'banana', 'cherry', 'date']
```

Explanation: Inside [], Python ignores newlines — the list keeps reading until the closing bracket.

Example 5: NameError from a typo

```
score = 95
print(Score)
```

Expected Output:

```
NameError: name 'Score' is not defined
```

Explanation: score (lowercase) was defined; Score (uppercase) was not. Python is case-sensitive.

Example 6: Listing the keywords

```
import keyword
print(keyword.kwlist)
```

Expected Output:

```
['False', 'None', 'True', 'and', 'as', 'assert', 'async', 'await', 'break', 'class',
'continue', 'def', 'del', 'elif', 'else', 'except', 'finally', 'for', 'from',
'global', 'if', 'import', 'in', 'is', 'lambda', 'nonlocal', 'not', 'or', 'pass',
'raise', 'return', 'try', 'while', 'with', 'yield']
```

Explanation: A quick way to remember which words you can't use as names.

4. Common Mistakes

Mistake	What Happens	Fix
Mixing tabs and spaces	TabError or weird IndentationError	Configure editor to insert spaces (VS Code: enabled by default for .py)
Forgetting : after if, for, def	SyntaxError: expected ':'	Add the colon
Using a keyword as variable: class = "math"	SyntaxError: invalid syntax	Rename to class_name or subject
Inconsistent indentation in same block	IndentationError: unexpected indent	Use 4 spaces everywhere
print vs Print	NameError	Lowercase — Python is case-sensitive

5. Practice Tasks

- Spot the error (3 min): Write the program from Example 5 (NameError demo) and read the error. Then fix it.
- Indent practice (5 min): Write an if that checks if $5 + 5 == 10$ and prints "math works" inside the block. Run it.
- Long line (5 min): Create a list of 8 of your favorite words across multiple lines using implicit continuation. Print it.

6. Key Takeaways

- One statement per line; no semicolons needed.
- Indentation defines blocks — 4 spaces, no tabs.
- Names follow rules: letters/digits/underscores, can't start with a digit, can't be a keyword.
- Python is case-sensitive.
- Read errors bottom-up — last line tells you what's wrong.

7. What's Next

In the next lesson we'll cover comments and code style — how to write Python that's not just correct, but readable.

Lesson 5: Comments, Indentation & Code Style

Module: 1 — Introduction to Python

Duration: 25 minutes

Difficulty: Beginner

Learning Objectives

By the end of this lesson, you will be able to:

- Write single-line and multi-line comments in Python.
- Apply consistent indentation (4 spaces) to all blocks.
- Follow the core PEP 8 style guidelines for names, spacing, and line length.
- Decide when a comment adds value and when it doesn't.

Prerequisites

- Lesson 4: Python Syntax Basics

1. Why This Matters

Code is read far more often than it's written. Style isn't decoration — it's how you (and others) understand your code six months from now. Python's official style guide, PEP 8, is widely followed, and most jobs expect you to know it. Good style also catches bugs: clear names and consistent spacing make mistakes visible.

2. Concept

2.1 Single-line Comments

Anything from `#` to the end of the line is a comment. Python ignores it.

```
# This whole line is a comment
x = 5    # comment after a statement (note 2 spaces before #)
```

Use comments to explain why code does something — not what it does (the code already shows that):

```
# BAD — restates the code
x = x + 1    # add 1 to x

# GOOD — explains motivation
x = x + 1    # offset by 1 because the API uses 1-based indexing
```

2.2 Multi-line Comments

Python has no formal multi-line comment syntax. There are two conventions:

Option A — many `#` lines (preferred):

```
# This is the first line of a longer explanation.
# It spans multiple lines because the logic below
# is non-obvious.
result = compute_something()
```

Option B — triple-quoted string (technically a string literal, not a comment):

```
"""
This is a string that nobody assigns to a variable.
Python evaluates it and throws it away.
"""
```

Use Option B only for docstrings (you'll see these in Module 4) at the start of a module, function, or class. For inline notes, use #.

2.3 Indentation Rules (PEP 8)

- Use 4 spaces per indentation level.
- Never mix tabs and spaces.
- Continuation lines (long lines wrapped onto a second line) should align logically:

```
total = (item_price
         + tax
         + shipping)
```

Most editors handle this for you. In VS Code, the Python extension auto-configures 4 spaces.

2.4 Naming Conventions

Item	Convention	Example
Variable	snake_case	user_name, total_score
Function	snake_case	calculate_tax(), read_file()
Constant	UPPER_SNAKE_CASE	MAX_USERS, PI
Class	PascalCase	Student, BankAccount
"Private"	leading _	_internal_helper

Use descriptive names. n and x are okay as loop counters; for anything else, spell it out:

```
# BAD
n = 86400

# GOOD
seconds_per_day = 86400
```

2.5 Spacing

Spaces around operators:

```
# GOOD
total = price * quantity + tax

# BAD
total=price*quantity+tax
```

One space after a comma:

```
# GOOD
print("a", "b", "c")
```

```
# BAD
print("a","b","c")
```

No space inside brackets:

```
# GOOD
items = [1, 2, 3]
```

```
# BAD
items = [ 1, 2, 3 ]
```

2.6 Line Length

PEP 8 suggests 79 characters maximum (some teams use 100). Long lines:

- Are hard to read on small screens or side-by-side diff views.
- Suggest the logic is too packed — break it up.

Most editors show a ruler at 80 chars. Use it.

2.7 Blank Lines

- One blank line between related functions (Module 4).
- Two blank lines between top-level functions and classes.
- Don't put blank lines at the top of a file or between every statement — that fragments the code.

2.8 The Zen of Python

A tongue-in-cheek style philosophy built into Python itself. Run it in the REPL:

```
import this
```

It lists 19 aphorisms. Three to remember now:

- Beautiful is better than ugly.
- Readability counts.
- There should be one — and preferably only one — obvious way to do it.

3. Code Examples

Example 1: Comments that help vs. comments that don't

```
# Calculate the total price including 18% GST tax.
# We add tax after any discounts have already been applied.
final_price = subtotal * 1.18
print(final_price)
```


Expected Output (with subtotal not defined — this is illustrative; replace subtotal to run):

(value depending on subtotal)

Explanation: The comment explains why (and the tax rate), not the multiplication itself.

Example 2: Good vs. bad naming

```
# GOOD
seconds_in_a_day = 60 * 60 * 24
print("Seconds per day:", seconds_in_a_day)

# BAD — what does s mean?
s = 60 * 60 * 24
print(s)
```

Expected Output:

```
Seconds per day: 86400
86400
```

Explanation: Both work, but only the first is readable a month from now.

Example 3: Proper spacing

```
price = 100
quantity = 3
discount = 0.1
total = price * quantity * (1 - discount)
print("Total:", total)
```

Expected Output:

```
Total: 270.0
```

Explanation: Operators have spaces around them, commas have spaces after them — easy to scan.

Example 4: The Zen of Python

```
import this
```

Expected Output:

```
The Zen of Python, by Tim Peters

Beautiful is better than ugly.
Explicit is better than implicit.
Simple is better than complex.
Complex is better than complicated.
Flat is better than nested.
Sparse is better than dense.
Readability counts.
Special cases aren't special enough to break the rules.
Although practicality beats purity.
```

```
Errors should never pass silently.
Unless explicitly silenced.
In the face of ambiguity, refuse the temptation to guess.
There should be one-- and preferably only one --obvious way to do it.
Although that way may not be obvious at first unless you're Dutch.
Now is better than never.
Although never is often better than now.
There are also a few stubborn tasks left to do.
If the implementation is hard to explain, a bad idea is, it's a bad idea.
If the implementation is easy to explain, it may be a good idea.
Namespaces are one honking great idea -- let's do more of them!
```

Explanation: Built into Python as an easter egg. The actual text is essentially these guiding principles.

Example 5: A well-styled small program

```
# Compute the average of three test scores.

score_1 = 85
score_2 = 92
score_3 = 78

average = (score_1 + score_2 + score_3) / 3

print("Average score:", average)
```

Expected Output:

```
Average score: 85.0
```

Explanation: Note: descriptive names, spaces around operators, one blank line between logical groups, comment explains the program's purpose.

4. Common Mistakes

Mistake	What Happens	Fix
Comments that restate code	Adds noise, no value	Comment the why, not the what
camelCase variable names	Works, but non-Pythonic	Use snake_case
Lines over 100 characters	Hard to read in diffs	Break into multiple lines
Inconsistent indentation (2 vs 4 spaces)	Looks scattered; possible errors	Pick 4 spaces and stick with it
Mixing tabs and spaces	TabError in Python 3	Configure editor to insert spaces

5. Practice Tasks

- Rewrite for style (5 min): Take this badly-styled snippet and fix it:

```
python X=10 y =20 PRINT( X+y )
```

- Comment a program (5 min): Add one why comment to your hello_me.py from Lesson 3.

- Zen (3 min): Run import this and pick the principle that surprises you most. Note it for discussion.

6. Key Takeaways

- # starts a comment to end-of-line. No real multi-line comment syntax.
- Comment the why, not the what.
- 4 spaces per indent level, no tabs.
- snake_case for variables/functions, PascalCase for classes, UPPER_SNAKE_CASE for constants.
- Spaces around operators, after commas, none inside brackets.
- PEP 8 is the reference — pip install pycodestyle later for automated checking.

7. What's Next

In the final lesson of this module, we'll consolidate how you can actually run Python — the REPL, scripts, and IDEs — so you know which tool to reach for in different situations.

Lesson 6: Running Python — REPL, Scripts & IDEs

Module: 1 — Introduction to Python

Duration: 30 minutes

Difficulty: Beginner

Learning Objectives

By the end of this lesson, you will be able to:

- Use the REPL effectively for quick experiments.
- Run Python scripts from the terminal with arguments.
- Use VS Code's debugger to step through a program.
- Decide which tool (REPL, script, notebook, IDE) fits a given task.

Prerequisites

- Lesson 5: Comments & Style
- VS Code installed with the Python extension (from Lesson 2).

1. Why This Matters

Python gives you several ways to run code — and using the right one for the job makes you dramatically faster. Beginners often try to write a full script when one REPL line would do, or wrestle with a debugger when a `print()` would tell them everything. This lesson is about choosing the right tool.

2. Concept

2.1 The REPL

The REPL is for trying things out. Open a terminal, type `python`, and you get the `>>>` prompt:

```
>>> 2 + 2
4
>>> "hello".upper()
'HELLO'
>>> len("python")
6
```

When to use it:

- Checking how a function behaves (`len`, `str.upper`, `round`).
- Testing a one-line expression before pasting it into a file.
- Inspecting a value you imported.

Limitations:

- Code disappears when you exit.

- Indenting multi-line blocks is awkward (you have to add ... continuation).
- Can't save your work easily.

Exit with `exit()` or `Ctrl+D` (mac/linux) / `Ctrl+Z` + `Enter` (Windows).

2.2 Scripts (.py files)

A script is a saved .py file. Run it with `python filename.py`.

When to use it:

- Any program that's more than 3-4 lines.
- Anything you want to share, version-control, or run again.
- Assignments, projects, the capstone — everything in this course beyond exploration.

You already did this in Lesson 3.

2.3 Passing arguments to a script

You can pass command-line arguments to your script and read them via `sys.argv`:

```
# greet.py
import sys
print("Hello, ", sys.argv[1])
```

Run:

```
python greet.py Alex
```

`sys.argv` is a list. The first element (`sys.argv[0]`) is the script name; the rest are the arguments. You'll use this in projects.

2.4 IDEs and Editors

Tool	Strengths	Best for
VS Code	Lightweight, free, extensions, integrated terminal & debugger	Daily use — this course
PyCharm Community	Heavier, refactoring tools, project navigation	Larger Python-only projects
Jupyter Notebook	Cells, inline plots, narrative	Data exploration, learning notebooks
IDLE	Bundled with Python, simple	Absolute beginners with no editor

For this course: VS Code unless you have a strong reason otherwise.

2.5 The VS Code Workflow

- Open VS Code.
- File → Open Folder → pick your project folder. (Always open the folder, not just one file — VS Code uses the folder as the workspace root.)

- Create a new .py file: Ctrl+N, then save with Ctrl+S as something.py.
- Press F5 to run with the debugger, or click the ► play button (top right) to run normally.
- The output appears in the integrated terminal (bottom panel).

2.6 Using the Debugger

The debugger lets you pause execution and inspect values. This becomes invaluable from Module 4 onward.

- Click in the gutter next to a line number — a red dot appears (a breakpoint).
- Press F5. Execution pauses at the breakpoint.
- Hover over any variable to see its current value.
- Use the toolbar: Step Over (F10), Step Into (F11), Continue (F5).

You don't need to master this in Module 1 — but try setting one breakpoint now so you've seen the workflow.

2.7 Jupyter Notebooks (preview only)

Jupyter is widely used in data science. A notebook is a .ipynb file with code cells and text cells mixed together. We won't use them in this course's core path, but you'll likely meet them in Module 12+ or in data work later.

To try one:

- Install VS Code's "Jupyter" extension.
- Create a file ending in .ipynb.
- Add a code cell with `print("hi")` and click ► next to the cell.

2.8 Choosing the right tool

Task	Right tool
Test what <code>round(3.567, 2)</code> returns	REPL
Build a calculator program	Script + VS Code
Compute a tax once and never again	REPL
Explore a CSV with charts	Jupyter notebook (later)
Track down why a function returns the wrong value	VS Code debugger

3. Code Examples

Example 1: REPL exploration

In the REPL:

```
>>> name = "World"
>>> message = f"Hello, {name}!"
>>> message
>>> len(message)
```

Expected Output:

```
'Hello, World!'  
13
```

Explanation: f"..." is an f-string (we'll cover them fully in Module 2). The REPL prints values automatically — no print() needed.

Example 2: A runnable script with arguments

Save as greet.py:

```
import sys  
  
if len(sys.argv) < 2:  
    print("Usage: python greet.py <name>")  
else:  
    print("Hello,", sys.argv[1])
```

Run:

```
python greet.py Alice
```

Expected Output:

```
Hello, Alice
```

And:

```
python greet.py
```

Expected Output:

```
Usage: python greet.py <name>
```

Explanation: sys.argv always has at least one element (the script name). Always check length before accessing extras.

Example 3: Multiple arguments

Save as add.py:

```
import sys  
  
a = int(sys.argv[1])  
b = int(sys.argv[2])  
print(a + b)
```

Run:

```
python add.py 5 7
```

Expected Output:

Explanation: `sys.argv` values are always strings. To do math, convert with `int()` (we'll cover type conversion in Module 2).

Example 4: Running a Python one-liner from the terminal

You don't always need a file:

```
python -c "print(2 ** 10)"
```

Expected Output:

```
1024
```

Explanation: `-c` runs the string as a Python program. Handy for quick shell scripts.

Example 5: A debugger-friendly script

Save as `buggy.py`:

```
def double(x):
    return x + x    # set a breakpoint on this line

result = double(7)
print("Result:", result)
```

Run with F5 in VS Code with a breakpoint on `return x + x`. Hover over `x`.

Expected Output (after F5 → Continue):

```
Result: 14
```

Explanation: With the breakpoint, you can confirm `x` is 7 before the function returns. Practice this once now so you remember it exists when you actually need it.

4. Common Mistakes

Mistake	What Happens	Fix
Running python in VS Code without opening a folder	Terminal opens in your home directory	File → Open Folder first
Forgetting to save before running	Old version runs	Enable Auto Save (File menu) or Ctrl+S first
Confusing the integrated terminal with the REPL	Typing <code>print(x)</code> in the REPL works; typing it in the terminal directly doesn't	Type <code>python</code> to start the REPL first
<code>sys.argv</code> items are strings, treating them as ints	<code>TypeError</code> on math	Wrap in <code>int()</code> or <code>float()</code>
Selected the wrong interpreter in VS Code	"Module not found" errors later	Ctrl+Shift+P → "Python: Select Interpreter"

5. Practice Tasks

- REPL session (5 min): Open the REPL. Compute (a) $7 * 8$, (b) the string `"python".count("p")`, (c) `round(3.14159, 2)`. Exit cleanly with `exit()`.
- Script with args (10 min): Write `times.py` that takes two numbers from the command line and prints their product. Test with `python times.py 6 7` → expect 42.
- Debugger drill (5 min): Open `greet.py` from Example 2 in VS Code. Set a breakpoint on the `else` line. Run with F5 and `Alice` as an argument. Hover over `sys.argv` and confirm what you see.

6. Key Takeaways

- REPL for one-off experiments; scripts for everything else.
- Run scripts with `python file.py`; pass arguments via `sys.argv`.
- `python -c "..."` runs a quick one-liner from the shell.
- VS Code + Python extension is the recommended setup for this course.
- The debugger pauses execution — invaluable as programs get larger.

7. What's Next

This is the end of Module 1. Next, Module 2: Variables & Data Types — where we stop just printing things and start storing, transforming, and reading them from the user.

Before moving on, complete the exercises, the quiz, the assignment, and the mini project.

Module 1 — Exercises

Three exercises per lesson, ramping from easy to hard. Solutions should be saved as .py files. Run each with `python filename.py`.

Lesson 1.1 — What is Python?

Exercise 1: Version check (Easy)

Task: Write a program that prints the current Python version using the `sys` module.

Sample Input: (none)

Expected Output (example, will vary):

```
3.11.5 (main, Sep  7 2023, 12:00:00) [MSC v.1936 64 bit (AMD64)]
```

Hint: `import sys` then `print sys.version`.

Exercise 2: Two facts (Medium)

Task: Write a program that prints two true statements about Python (one per line) — one about its creator and one about its year of release.

Sample Input: (none)

Expected Output:

```
Python was created by Guido van Rossum.  
Python was first released in 1991.
```

Exercise 3: Compare languages (Hard)

Task: Write a program that prints a table comparing Python and one other language you know of. Include columns: language, typing (static/dynamic), compiled or interpreted.

Expected Output (sample):

Language	Typing	Execution
Python	Dynamic	Interpreted
C	Static	Compiled

Hint: Use multiple `print()` calls with `\t` for tab separation, or align with spaces.

Lesson 1.2 — Installation

Exercise 1: Print pip path (Easy)

Task: In your terminal, run `pip --version` and copy the output into a .py file as a comment at the top.

Hint: Comments start with `#`. The file does not need to print anything.

Exercise 2: Version check from inside Python (Medium)

Task: Write a program that imports sys and prints whether the version is 3.10 or newer.

Expected Output:

```
Python 3 detected. Major: 3, minor: 11
Modern enough for this course.
```

Hint: Use sys.version_info.major and sys.version_info.minor.

Exercise 3: Path inspection (Hard)

Task: Write a program that prints (a) the Python executable path and (b) the platform you're running on.

Expected Output (example):

```
Executable: C:\Python311\python.exe
Platform: win32
```

Hint: sys.executable and sys.platform.

Lesson 1.3 — Your First Program

Exercise 1: Greeting (Easy)

Task: Print Hello, [your name]! exactly once.

Expected Output:

```
Hello, Priya!
```

Hint: One print() call.

Exercise 2: ASCII Box (Medium)

Task: Print a box around the word "PYTHON" using # characters.

Expected Output:

```
#####
# PYTHON #
#####
```

Exercise 3: Receipt (Hard)

Task: Print a mock receipt with three items and their prices, plus a total line.

Expected Output:

```
Apple      50
Bread      40
Milk       60
-----
Total      150
```

Hint: Use sep or pad strings with spaces. Multiple print() calls.

Lesson 1.4 — Syntax Basics

Exercise 1: Fix the error (Easy)

Task: Copy this snippet, run it, fix the error, and explain in a comment what was wrong:

```
score = 95
print(Score)
```

Expected Output (after fix):

```
95
```

Exercise 2: Keywords (Medium)

Task: Write a program that prints how many reserved keywords Python has.

Expected Output:

```
Python has 35 reserved keywords.
```

Hint: import keyword then len(keyword.kwlist).

Exercise 3: Indent or fail (Hard)

Task: Without copying, write a program that uses an if True: block containing two print() statements. Both must print correctly.

Expected Output:

```
First inside if.
Second inside if.
```

Hint: Indent both lines by 4 spaces under the if True:.

Lesson 1.5 — Style

Exercise 1: Add a why-comment (Easy)

Task: Take the program below, add a single comment explaining the + 1, and run it.

```
month_index = 5
human_month = month_index + 1
print(human_month)
```

Expected Output:

```
6
```

Exercise 2: Rename for clarity (Medium)

Task: Rewrite this with PEP 8 names:

```
X = 8
Y = 6
A = X * Y
print(A)
```

Expected Output (unchanged):

```
48
```

Exercise 3: Style audit (Hard)

Task: Rewrite the snippet below to follow PEP 8 (spacing, naming, blank lines, comments):

```
TotalPrice= 100 *3+20
PRINT(TotalPrice)
```

Expected Output:

```
320
```

Hint: print is lowercase; use snake_case; add spaces around * and +.

Lesson 1.6 — Running Python

Exercise 1: Arg echo (Easy)

Task: Write echo.py that prints the first command-line argument.

Sample Input:

```
python echo.py hello
```

Expected Output:

```
hello
```

Hint: sys.argv[1].

Exercise 2: Multiply via args (Medium)

Task: Write mul.py that takes two numbers as command-line arguments and prints their product.

Sample Input:

```
python mul.py 8 9
```

Expected Output:

```
72
```

Hint: Convert with int().

Exercise 3: Safe greeting (Hard)

Task: Write safe_greet.py that:

- If 1 argument is passed, prints Hello, [arg]!
- If no arguments are passed, prints Hello, stranger!
- If more than 1 argument is passed, prints Too many names!

Sample Inputs and Expected Outputs:

```
python safe_greet.py Alex  
Hello, Alex!
```

```
python safe_greet.py  
Hello, stranger!
```

```
python safe_greet.py Alex Bob  
Too many names!
```

Hint: `len(sys.argv)` — remember `argv[0]` is the script name.

Module 1 — Quiz

10 multiple-choice questions covering Lessons 1-6. Pick one answer per question, then check against the explanations.

Question 1

Who created Python?

A. Linus Torvalds B. Guido van Rossum C. James Gosling D. Bjarne Stroustrup

Answer: B

Explanation: Guido van Rossum created Python and released it in 1991. (Lesson 1)

Question 2

Which of these is not a feature of Python 3?

A. print is a function B. Strings are Unicode by default C. $5 / 2$ gives 2 D. $5 / 2$ gives 2.5

Answer: C

Explanation: In Python 3, $/$ always returns a float. $5 / 2$ is 2.5. The 2 result is Python 2 behavior. (Lesson 1)

Question 3

Which command verifies Python is installed correctly from the terminal?

A. python install B. python --version C. pip python D. version python

Answer: B

Explanation: python --version (or python3 --version on macOS/Linux) prints the installed version. (Lesson 2)

Question 4

What will this code print?

```
print("hello", "world", sep="-")
```

A. hello world B. hello-world C. helloworld D. hello, world

Answer: B

Explanation: sep changes the separator placed between the arguments. Default is a space; here we set it to "-". (Lesson 3)

Question 5

Which line is invalid Python syntax?

A. x = 5 B. name = "Alice" C. 2name = "Alice" D. _name = "Alice"

Answer: C

Explanation: Identifiers cannot start with a digit. The other three are valid. (Lesson 4)

Question 6

What error does this code produce?

```
if True:
    print("hi")
```

A. SyntaxError B. IndentationError C. NameError D. No error — prints "hi"

Answer: B

Explanation: Python expects an indented block after the colon on an if statement. (Lesson 4)

Question 7

Which of the following is not a Python keyword?

A. pass B. print C. lambda D. yield

Answer: B

Explanation: print is a built-in function, not a reserved keyword. You can technically reassign it (don't!). The others are reserved. (Lesson 4)

Question 8

According to PEP 8, what is the preferred naming convention for a Python variable?

A. camelCase (totalPrice) B. PascalCase (TotalPrice) C. snake_case (total_price) D. UPPER_CASE (TOTAL_PRICE)

Answer: C

Explanation: PEP 8 uses snake_case for variables and functions, PascalCase for classes, and UPPER_SNAKE_CASE only for constants. (Lesson 5)

Question 9

You want to test what round(3.14159, 2) returns. Which tool is the most efficient?

A. Write a full .py file and run it B. Use the REPL C. Set up Jupyter Notebook D. Use VS Code's debugger

Answer: B

Explanation: The REPL is built for one-line experiments. You'd open Python, type the expression, and see the result. (Lesson 6)

Question 10

What does `sys.argv[0]` always contain when you run `python myscript.py hello world`?

A. hello B. world C. `myscript.py` (the script name) D. An empty string

Answer: C

Explanation: `sys.argv[0]` is always the script's name. Actual arguments start at index 1. (Lesson 6)

Scoring

- 9-10 correct: Excellent — move on to the assignment and project.
- 7-8 correct: Solid — quickly review the lessons for the ones you missed.
- 5-6 correct: Re-read the lessons referenced in the explanations.
- Below 5: Repeat Module 1 lessons before the assignment. Don't skip.

Module 1 — Assignment: Personal Info Card

Estimated time: 45-60 minutes

Combines concepts from: Module 1 (all lessons)

Brief

Build a Python program that prints a formatted personal information card for yourself. You'll exercise everything from this module: writing and running a script, using `print()`, formatting output, following code style, and adding meaningful comments.

This is intentionally small — the point isn't difficulty, it's clean execution. You'll be surprised how often "clean execution" matters more than cleverness in real jobs.

Requirements

Your program must:

- Define at least 6 variables with this information:
- `full_name` (string)
- `age` (integer)
- `city` (string)
- `email` (string)
- `favorite_language` (string — your guess; "Python" is fine)
- `years_coding` (integer — 0 is fine)
- Print a formatted card that:
- Has a clear top and bottom border made of `#` characters.
- Centers or aligns the heading "PERSONAL INFO CARD".
- Lists every field on its own line in the format `Label : Value`.
- Include a comment at the top of the file with:
- The assignment name.
- Your name.
- The date you wrote it.
- Follow PEP 8:
- `snake_case` variable names.
- Spaces around `=` in assignments at top level.
- One blank line between logical sections.

Constraints

- Use only concepts from Module 1.
- No `input()` (that's Module 2). All info is hard-coded as variables.
- No loops or `if` statements (Module 3).
- No functions (Module 4).
- No external libraries.

- Maximum line length: 80 characters.

Expected Behavior

When you run `python assignment_module1.py`, output should look like this (your details, of course):

```
#####
#   PERSONAL INFO CARD   #
#####
Name           : Priya Sharma
Age            : 22
City           : Bengaluru
Email          : priya@example.com
Favorite Language: Python
Years Coding   : 1
#####
```

The exact spacing doesn't have to match to the character — but the structure (borders, heading, label/value rows) must be present and readable.

Starter Code

Save this as `assignment_module1.py` and fill it in:

```
# Module 1 Assignment: Personal Info Card
# Author: [Your Name]
# Date:   [Today's date]

# --- Personal data ---
full_name = ""
age = 0
city = ""
email = ""
favorite_language = ""
years_coding = 0

# --- Print the card ---
print("#####")
# TODO: add the rest of the card.
```

Hints

- To print a label and value separated by a `:`, use `sep`:

```
python print("Name", "Alex", sep=" : ")
```

- To pad a label to a fixed width, you can hard-code spaces:

```
python print("Age      :", age)
```

We'll see proper string formatting in Module 2 — for now, hard-coding is fine.

- Print the borders the same way as the heading line so widths match.

Grading Rubric

Criterion	Weight	Notes
All 6 variables defined	20%	Right types matter — age should not be a string
Output renders cleanly	30%	Borders align, fields are readable
Comment header present	10%	Name, assignment, date
PEP 8 style	25%	snake_case, spacing, line length
Code is runnable as-is	15%	python assignment_module1.py works without edits

Total: 100% — passing threshold 70%.

Submission

- Save your file as `assignment_module1.py`.
- Make sure it runs with `python assignment_module1.py` from the terminal.
- Take a screenshot of the terminal output.
- If you're using a code review tool / GitHub, push the file. Otherwise, keep both the file and the screenshot in a `/submissions/module-01/` folder.

Self-Check Before Submitting

- ☐ Does the file run without errors?
- ☐ Does the output have borders, a heading, and 6 labeled fields?
- ☐ Are the variable names in snake_case?
- ☐ Are there spaces around `=`?
- ☐ Is there a comment header with your name and the date?
- ☐ Are all lines under 80 characters?

Module 1 — Mini Project: Interactive Calculator (Module-1 Edition)

Overview

Build a calculator script that performs and displays the result of a fixed set of arithmetic operations using only the concepts you've learned in Module 1. The numbers are hard-coded at the top of the file (real interactive input comes in Module 2). The goal is a clean, readable script you can show as proof you can build a working program end-to-end.

Concepts Used

- Saving and running a .py file (Lesson 3)
- The print() function with sep / end (Lesson 3)
- Arithmetic expressions (Lesson 3)
- Variables and snake_case naming (Lessons 4-5)
- Comments and PEP 8 style (Lesson 5)

No loops, conditionals, functions, or input() — those come later.

Features

Your calculator must:

- Hard-code two numbers at the top: num_a and num_b.
- Compute and display:
 - Sum ($a + b$)
 - Difference ($a - b$)
 - Product ($a * b$)
 - True division (a / b)
 - Integer division ($a // b$)
 - Remainder/modulo ($a \% b$)
 - a raised to the power of b ($a ** b$)
- Display the result as a formatted "report" with a heading and borders.

Starter Code

Save as calculator.py:

```
# Mini Project: Interactive Calculator (Module 1 edition)
# Author: [Your Name]

# --- Inputs (change these to test different numbers) ---
num_a = 12
num_b = 5

# --- Report header ---
print("=" * 40)
print("          CALCULATOR REPORT")
print("=" * 40)
```

```

print("Operand A:", num_a)
print("Operand B:", num_b)
print("-" * 40)

# --- Operations ---
# TODO: print each operation in the format below.
# Example:
# print("Sum          :", num_a + num_b)

# --- Footer ---
print("=" * 40)

```

You can use `"=" * 40` (a string repeated 40 times) — this is just shorthand and doesn't introduce new concepts.

Expected Interaction

When you run `python calculator.py` with the starter values:

```

=====
                        CALCULATOR REPORT
=====
Operand A: 12
Operand B: 5
-----
Sum          : 17
Difference    : 7
Product       : 60
Division      : 2.4
Integer Div   : 2
Remainder     : 2
Power (A^B)   : 248832
=====

```

Change `num_a` and `num_b` at the top of the file and re-run. The output should update.

Step-by-Step Approach

- Set up the file. Save the starter code as `calculator.py`.
- Print one operation first. Pick "Sum" — confirm it prints correctly before adding the rest.
- Add the remaining six lines one at a time. After each, run the script.
- Adjust spacing. Make sure all labels align — count spaces if you have to.
- Test with different numbers. Try `num_a = 7`, `num_b = 3`. Try `num_a = 100`, `num_b = 10`. Confirm output makes sense.
- Style pass. Make sure variable names are `snake_case`, all comments are accurate, no line is over 80 characters.

Sample Variations

Try changing `num_a` and `num_b` to confirm your script handles different cases:

- `num_a = 15`, `num_b = 4` → Sum=19, Difference=11, Product=60, Division=3.75, etc.

- `num_a = 7, num_b = 7` → Difference=0, Remainder=0
- `num_a = 9, num_b = 2` → Power=9 ** 2=81

> Don't try `num_b = 0` — division by zero raises `ZeroDivisionError`. We'll handle that properly in Module 8 (Error Handling). For now, just don't use zero as the second number.

Extension Challenges

Tackle these once the base version works. They still stay within Module 1 concepts.

- Two-column layout — Print the operations in two columns side-by-side using `sep` or hard-coded spacing.
- Multi-set report — Create a second pair of variables `num_c` and `num_d` and print a second calculator report below the first one (just copy the print block).
- Themed borders — Replace the `=` and `-` characters with a custom pattern like `~~` to make your report look different.

Grading Rubric

Criterion	Weight
Script runs without errors	25%
All 7 operations printed correctly	30%
Output is formatted and readable	20%
PEP 8 style applied	15%
Clear, accurate comments	10%

Total: 100% — passing threshold 70%.

Reflection

After finishing, ask yourself:

- What would you need to change to let the user type in `num_a` and `num_b` instead of hard-coding them? (Preview of Module 2.)
- How would you protect against `num_b = 0`? (Preview of Modules 3 and 8.)
- How would you avoid copy-pasting the print line seven times? (Preview of Module 4: functions.)

You don't need answers now — just notice the questions. Each one is a doorway to the next module.

Submission

- Save your final file as `calculator.py`.
- Take a screenshot of one run.
- If your course platform requires it, push to a `/submissions/module-01/` folder. Otherwise keep it locally for your portfolio.